运动规划节点（n_moveit）端口对正与势场力接入方案

状态空间 · 运动规划 · 2026-11-01

版本：v1.0 · 2026-11-01 · 提交 907c3528

范围：状态空间画布「运动规划」节点及其上下游（原子技能层 → 运动规划 → 执行器）

性质：内部技术方案与验收记录（含实测数字与回读命令）

---

1. 结论先行

上游接线	8 个原子技能 out1 接在 in1（目标点）/ in2（安全约束）	8 个技能各占 in4~in11，一技能一端口	画布 219 条连线回读	
技能出值	实现 `return True` ⇒ out1/out2 恒空	out1 = 技能动作帧（含 target，mm）；out2 = 过闸结果	总线话题实读	
势场力	in3 有线但节点未声明、实现不读（装饰线）	in3 真读：`	F	≥ 阈值` 拒规划，否则沿 −F 有界偏置目标点	输出 `force` / `force_bias_mm`
安全约束	下游写死 50/20 mm	in2 读安全边界节点 out3 = 执行器守卫 GATE 同源	输出 `limits`	
执行方式	`PURE=False` ⇒ 直通，实现从不被调用	`pure_verified=true` ⇒ `PURE=True`，实测 status=实现执行	中间件 status + 段表	
单拍代价	探针 50.7 ms/次（每拍跑）	封 TTL 缓存后整拍 0.08 ms	30 次循环计时	

自检：`tools/ss_plan_ports_selfcheck.py` **16 项全通过**。

---

2. 问题与根因

2.1 端口语义错位（接线错）

画布原连线把 8 个原子技能的 `out1`（技能动作）接到运动规划的 `in1` 与 `in2`。而运动规划实现的口径写在 `src/lerobot/engineering/nodes/plan_impl.py` 头注释里：

```
in1 = 目标点  {"tcp":[x,y,z]} / {"pos":[..]} / [x,y,z] / 点名 / 槽号 / 观测点
in2 = 安全约束 {"max_step_mm":50,"max_down_mm":20}
```

即：技能动作被当成坐标去解析。即便技能真出值，规划只会报「目标点取不到」。

2.2 上游出值为空（线在值没有）

`library.py::node_ss_atomic` 原来只 `return True`。中间件壳的约定是「实现返回 dict 才映射到端口」，非 dict ⇒ 判为直通或丢弃 ⇒ `sssk*.out1/out2` 永远是 `None`。这是「接了等于没接」的典型形态：源码里读取代码齐全、画布上有连线，实测值却是空。

2.3 势场力是装饰线

`ss_grav_h.out1 → n_moveit.in3` 一直画在图上，但节点 `inputs` 只声明 `['in1','in2']`，实现里也没有任何 `in3` 读取。中间件按 `spec.inputs` 逐口取数 ⇒ in3 永远拿不到值。

2.4 执行面是直通

`src/lerobot/middleware/nodes/n_moveit.py` 生成时 `PURE=False`，壳子第一句就 `return _passthrough(ctx)`，真实现一次也没被调用过。实测状态字符串：`直通（PURE=False: 只走数据面）`。

---

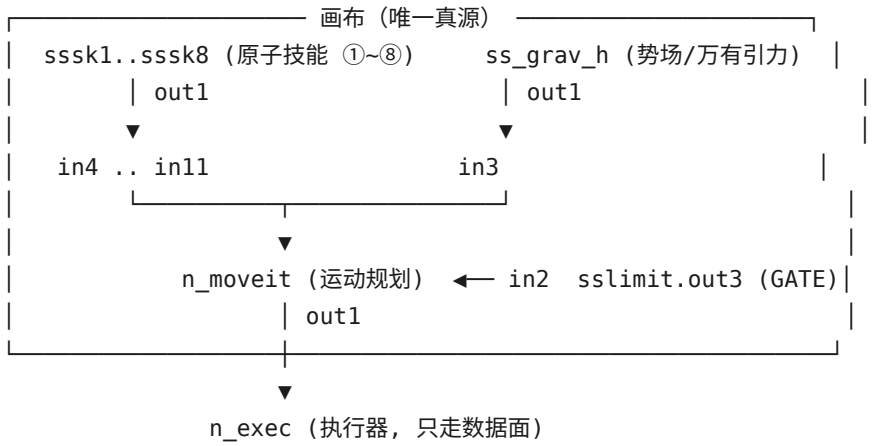
3. 目标

1. **端口对正**：按实现里写明的语义分配端口，不按节点名猜。
2. **真读势场力**：势场不再是旁路数字，参与目标点决策并能否决规划。
3. **真执行**：中间件实际调用实现，输出进总线、进执行器节点。
4. **可测试**：一条命令跑完全部判据，含否决分支与零回归。
5. **可观察**：每拍输出足以判断「读到了什么、为什么这么规划」。

6. **可配置**：势场三项参数进工程配置真源，参数中心可改、热生效。

---

4. 总体数据流



每拍（画布 loop 10Hz）：`pull()` 按端口取总线最新值 → `compute()` → `plan_impl.run(ctx)` → `push()`

把 `out1` 发回总线并转发给 `n_exec.in1`。

---

5. 端口契约

5.1 运动规划 n_moveit

in1	目标点（显式）	当前无上游（预留：工程点名 / 槽号 / 观测点 / 裸坐标）	优先
in2	安全约束	`sslimit.out3`	`max_step_mm` / `max_down_mm`，缺项回落 50/20
in3	势场力	`ss_grav_h.out1`	`dir`(单位向量) + `mag`(m/s 当量)；也认 `F` 向量
in4..in11	①~⑧ 原子技能动作	`sssk1..8.out1`	一技能一口，按 `(active, t)` 选一个
out1	规划结果	→ `n_exec.in1`	段表 + 取证块

为什么一技能一口：多个上游写同一个入口，中间件是「后写覆盖」。实测把 8 个技能合流到 in4 时，in4 里抓到的是 `safety_limit`、`l3_nominal`，技能帧被冲掉。分端口后每帧独立，选择逻辑落在下游，可取证。

5.2 原子技能 sssk1..8

out1	`{kind:"skill_act", skill:"SK01", stage:"接近", active:bool, target:[x,y,z] mm, target_unit, u:[4] m/s, gate:{…}, name, t}`
out2	`{kind:"skill_gate", skill, stage, f_need_N, u_gate, why, err_mm, saturated, t}`

无引擎轨迹时不返回 `False`，而是返回**带类型的空帧**：

```
{"kind":"skill_act","skill":"SK01","stage":"接
近","active":false,"target":null,"reason":"no_engine_trace"}
```

区分两种空：`active=false / reason=no_engine_trace` = 技能没激活；端口无任何帧 = 没接线。

5.3 运动规划 out1 取证字段

`start` / `start_age_s`	真机 TCP 起点（mm）与位龄
`skill`	选中技能：`skill / stage / port / n_frames / active_ports / f_need_N / u_norm`
`force`	势场：`mag / vec / main / layers`
`force_bias_mm`	偏置：`bias / norm_mm / scale_mm_per_unit / cap_mm / goal_raw / goal_biased / why`
`limits`	生效的安全约束（来自 in2）
`goal / segments / n / total_mm`	规划结果
`moveit`	可用性探针（带 `cached_age_s`）
`reason`	拒出段表的原因：`tcp_stale` / `force_veto` / `obs_unverified` / `low_conf` …

---

6. 势场力接入

6.1 输入语义

`ss_grav_h.out1 = {kind:"force_corrected", dir:[单位向量], mag:速度当量 m/s, main:主力层, layers:{L2,L3,L4,L5}, allow}`。

6.2 两个作用

① **否决**（安全优先）

```
|F| ≥ force_veto_mag  ⇒  ok=false, reason=force_veto, segments=[]
```

高势区不出段表，而不是硬规划过去。

② **有界偏置**（避让）

```
bias_vec = −F_vec × force_bias_mm_per_unit          # 沿合力反方向退让
若 ‖bias_vec‖ > force_bias_max_mm ⇒ 等比缩放到上限
goal_plan = goal + bias_vec
```

偏置只改目标点，不改步长与安全约束；偏置量原样进 `force_bias_mm` 取证。

6.3 实测

正常	mag=0.07125，dir=[0.2635,−0.3319,−0.1309]	bias=[−0.939, 1.182, 0.466] mm，‖bias‖=1.58 mm（≤20），段表 17 段
超标	mag=1.200（阈值 1.0）	ok=false，reason=force_veto，segments=0
现场	mag=0.1192 / 0.1485（L2 主力）	真进 `out1.force`，不参与否决

---

7. 安全约束同源

原来 `plan_impl` 里写死 `DEF_STEP=50mm / DEF_DOWN=20mm`。现改为读 `in2`，而 in2 由安全边界节点 `safety_limit_impl` 的 `out3` 提供，内容取自 `src/lerobot/arm/arm_control.py::GATE`：

```
{"max_step_mm":50.0,"max_down_mm":20.0,"require_idle":true,"require_power_on":true,
```

```
  "src":"arm_control.GATE (执行器守卫同一口径)"}
```

规划与执行器守卫用同一份数字，不存在「规划允许 50mm、执行器拒 50mm」的口径漂移。

实测 in2 取值：`{'max_step_mm': 50.0, 'max_down_mm': 20.0, ..., src=arm_control.GATE}`；

把 in2 改成 20/5 后段数从 17 变 40（真读，不是摆设）。

---

8. 真执行机制

中间件壳的判据只有一句：`PURE` 为真才调用实现。

① 画布节点 `params.pure_verified = true` 画布回读该字段

② `tools/gen_middleware_nodes.py` 重生成 壳里 `PURE = True`

③ `--check` 幂等校验 ✅ □□□□□□□□ JSON 一致 (80 个)

④ 跑一拍，读节点 status `实现执行`（而非 `直通 (PURE=False …)`）

本次对 `n_moveit`、`sssk1~8`、`sslimit` 共 10 个节点置位。其中运动规划本身不下发动作（`plan_impl` 不 import 任何下发接口、不写 exec 文件、不碰授权），执行器节点 `n_exec` 仍为 `PURE=False`（只走数据面）。

---

9. 工程配置参数

真源：`config/calib/zmax_calib.json` → `plan_node`

`force_bias_mm_per_unit`	50.0	mm/(m/s)	0~50 0	势场每 1 m/s 当量把目标点推开的毫米数； 改大→绕得更远更保守
`force_bias_max_mm`	20.0	mm	0~100 0	单次规划最大偏置；0 = 不偏置
`force_veto_mag`	1.0	m/s	当 0~10 `	F ≥ 该值` 拒规划；0 =

量 关闭否决

- 热生效：`plan_impl::_pcfg()` 按 mtime 重载，改文件下一拍即生效，不用重启。
- 参数中心：`tools/param_registry.py` 新增 `_PLAN_SPEC`，三项带中文名/单位/范围/影响，与真源同源。

- 复核：`python3 tools/param_registry.py list | grep plan_node`

---

10. 可测试性

```
PYTHONPATH=src gui-venv311/bin/python tools/ss_plan_ports_selfcheck.py
```

起点用临时文件注入（`ts=now`），不伪造现场结论；真机在线与否不影响端口判据。

A	中间件壳走实现（PURE=True）	status=实现执行
A'	壳回传 out1 是段表	n=17
B	in1 显式目标出段表	17 段 / 494.9 mm / src=in1
C	in4..in11 技能 target 出段表	17 段 / src=in4(技能 target) / SK03
C'	技能字段进输出	f_need_N=0.163
D	in3 势场真读	mag=0.07125 / main=L3
D'	目标点被偏置	bias=[−0.939,1.182,0.466]，1.58 mm
D''	偏置有界	1.58 ≤ 20 mm
D'''	偏置后仍出段表	n=17
E	势场过大拒规划	reason=force_veto / segments=0
F	in2 安全约束真读	20/5 → 40 段（vs 50/20 → 17 段）
G	配置三项可读	3 键齐
I	8 端口 8 帧全收到	n_frames=8
I'	选中 active 的那个	port=in8 / SK05
J	探针带缓存	moveit.cached_age_s 存在

H	无势场无技能时零回归	段表 abs_mm 逐位一致

结果：**16 通过 / 0 失败**。

---

11. 可观察性

每拍输出　总线话题 `ss.node.n_moveit.out1`（转发到 `ss.node.n_exec.in1`）

输入实况　`ss.node.n_moveit.in2/in3/in4..in11`

节点状态　节点 `last_status`（实现执行 / 直通 / 实现异常）

规划日志　`[plan □□] 段表 N 段 / Xmm · □□□□ …` 或 `势场力过大 … ⇒ 不出段表`

配置　`config/calib/zmax_calib.json:plan_node` + 参数中心

验证时注意：在跑的画布 loop 会往同一批 DDS 话题写旧值。进程内验证必须加 `ZMAX_MW_BUS=mem` 隔离，否则读到的是 loop 的旧帧（表现为「改对了但值还是旧的」）。

---

12. 性能

`plan_impl.run()` 的主要开销曾是 MoveIt 可用性探针 `MoveItPlan.plan()`：

探针单次	50.7 ms	命中缓存不调用
`run()` 单拍	50.8 ms	**0.08 ms**
缓存 TTL	—	`ZMAX_PLAN_MOVEIT_TTL_S`，默认 10 s，输出带 `cached_age_s`

画布 loop 预算 100 ms/拍（既有 220~310 ms 超预算为 80 节点老账，与本节点无关）。

若不封缓存，运动规划一家就吃掉半拍预算。

---

13. 变更清单

`src/lerobot/engineering/nodes/plan_impl.py`	新增 `_pcfg/_skill_act/_force/_moveit_probe`；run 增 in2/in3/in4..in11 读取、势场否决与偏置、取证块
`src/lerobot/engineering/nodes/library.py`	`node_ss_atomic` 返回 `{out1,out2}`（含无轨迹空帧）
`src/lerobot/engineering/nodes/safety_limit_impl.py`	安全边界分支新增 `out3` = GATE 限值
`config/calib/zmax_calib.json`	新增 `plan_node`（3 项）
`tools/param_registry.py`	新增 `_PLAN_SPEC`，`_dyn_extra` 支持 `plan_node.*`
`data/database/zmax/sources/canvas/state_space_obs.json`	10 条连线改口 + 1 条装饰线删除（220→219）+ 10 节点 `pure_verified`
`src/lerobot/middleware/nodes/*.py`	`gen_middleware_nodes.py` 重生成（80 节点，`--check` 幂等）
`tools/ss_plan_ports_selfcheck.py`	新增，16 项判据

提交：`907c3528`（已推送 origin/main）

---

14. 回滚

```
cp
/home/ubuntu/zmax/data/database/zmax/sources/canvas/_archive/state_space_obs_before_moveit_port
_align_v2_20261101_121150.json \
   /home/ubuntu/zmax/data/database/zmax/sources/canvas/state_space_obs.json
cd /home/ubuntu/zmax && ./gui-venv311/bin/python tools/gen_middleware_nodes.py
git revert 907c3528        # 代码侧回滚
```

第一版（v1：技能合流 in4）备份在同目录 `…_before_moveit_port_align_20261101_120912.json`。

---

15. 未完成项与验收条件

画布 loop 按新 未生效 `zmax-ss-canvas-loop` 仍在旧内存图，需重启（按规矩不自行重启）

图跑

真机段表	未验证	TCP 真值 152073 s 陈旧（42 小时前），规划按设计拒出段表 `reason=tcp_stale`
势场偏置的真机效果	未验证	需真机在线 + 势场在线，观察偏置后段表与实走偏差
in1 目标点上游	预留	当前目标由技能 in4..in11 提供；若要视觉点直驱，接 `ssyolo.out1`（plan 认 `{"obs": …}` 形式，需扩一处解析）

验收条件：重启后跑 `ss_plan_ports_selfcheck.py` 仍 16/16；`n_moveit` status=实现执行；

真实 TCP 新鲜时 `out1.n > 0` 且 `out1.skill.n_frames = 8`。

附录 A 命令速查

```
cd /home/ubuntu/zmax

# 端口自检 (16 项)
PYTHONPATH=src gui-venv311/bin/python tools/ss_plan_ports_selfcheck.py

# 中间件生成物一致性
./gui-venv311/bin/python tools/gen_middleware_nodes.py --check

# 现场一拍 (隔离在跑的 loop)
ZMAX_MW_BUS=mem PYTHONPATH=src gui-venv311/bin/python - <<'PY'
import sys; sys.path.insert(0,'src')
from lerobot.middleware import load_project
p=load_project(); p.step()
tr=p.nodes['n_moveit'].step()
o=(tr.get('out') or {}).get('out1') or {}
print(tr['status'], o.get('n'), (o.get('skill') or {}).get('n_frames'), (o.get('force') or
{}).get('mag'))
PY

# 参数中心
python3 tools/param_registry.py list | grep plan_node
```

附录 B 字段字典（技能帧）

`kind`	str	固定 `skill_act`
`skill`	str	模板码 SK01~SK08

`stage`	str	阶段中文名（接近/对位/下降/抓取/抬起/转移/插入/完成）
`active`	bool	当前阶段是否等于该技能阶段
`target`	list[3]	阶段目标点，**mm**（引擎内部为 m，节点处 ×1000）
`u`	list[4]	执行速度向量 m/s（与引擎 `u_exec_vec` 同口径）
`gate`	dict	过闸结果：`u / err_norm / why / f_need_N`
`t`	float	帧时间戳